

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

SmartLibrary Pro

Réalisé par :

Abdeljalil ELGADI
Hicham ABADOUR

Encadrante :

Mme Douae EL HILA

Module : Développement Java IHM

Année universitaire : 2025/2026

Remerciements

Avant tout, nous tenons à exprimer notre profonde gratitude à notre encadrante, **Mme Douae El Hila**, pour son accompagnement précieux, sa disponibilité constante, ses conseils pertinents et son soutien tout au long de la réalisation de ce projet. Son encadrement, ses orientations et son expertise nous ont permis de mener à bien ce travail dans les meilleures conditions.

Nous adressons également nos sincères remerciements à **Mme Ilhame El Farissi**, enseignante du module *Développement Java IHM*, pour la qualité de son enseignement, son engagement pédagogique et les connaissances qu'elle nous a transmises. Les compétences acquises au cours de cette formation ont constitué une base essentielle pour la conception et le développement de notre application.

Nous adressons nos sincères remerciements à notre chef de filière, **Monsieur Toumi Bouchentouf**, pour son engagement constant au sein de la filière, la qualité de son encadrement pédagogique ainsi que les efforts qu'il déploie afin d'assurer aux étudiants une formation de qualité. Son accompagnement, sa disponibilité et son implication contribuent grandement à notre réussite académique et au développement de nos compétences professionnelles. Nos remerciements s'adressent également à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet, par leurs conseils, leurs remarques constructives ou leur soutien.

Enfin, nous exprimons notre profonde reconnaissance à nos familles et à nos proches pour leur soutien moral, leur patience, leurs encouragements et leur confiance tout au long de notre parcours universitaire.

La réalisation de ce projet nous a permis de mettre en pratique les connaissances acquises dans les domaines du développement Java, de JavaFX, de JDBC, de MySQL ainsi que de l'architecture MVC et du patron DAO. Cette expérience a constitué une opportunité enrichissante pour renforcer nos compétences techniques, notre esprit d'analyse et notre capacité à travailler en équipe.

Table des matières

Remerciements	1
Table des figures	4
Liste des tableaux	5
Introduction	6
1 Présentation du projet	7
1.1 Description de l'application	7
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	8
2 Conception	9
2.1 Architecture MVC	9
2.2 Diagramme de classes UML	11
2.3 Diagramme de cas d'utilisation	11
2.4 Diagramme de séquence	14
3 Environnement de travail	15
3.1 Outils et technologies utilisés	15
3.2 Structure du projet	15
4 Répartition des tâches	18
4.1 Tâches de l'étudiant 1 – Abdeljalil ELGADI	18
4.2 Tâches de l'étudiant 2 – Hicham Abadour	20
5 Explication du code	24
5.1 Connexion à la base de données – Database.java	24
5.2 Les classes modèles	25
5.3 Les classes DAO	27
5.4 Les contrôleurs	28
5.5 Les fichiers FXML	29
5.6 Fonctionnalité spécifique au projet	30

6 Interfaces de l'application	32
6.1 Fenêtre principale	32
6.2 Interfaces de gestion de l'entité 1	32
6.3 Interfaces de gestion de l'entité 2	33
6.4 Tableau de bord – Statistiques	34
6.5 Menus et dialogues	34
6.6 Export de données	35
 Conclusion	 37
 Références	 38

Table des figures

2.1	Architecture MVC de l'application SmartLibrary Pro	10
2.2	Diagramme de classes UML	11
2.3	Diagramme de cas d'utilisation	13
2.4	Diagramme de séquence : ajout d'un livre	14
6.1	Fenêtre principale de l'application	32
6.2	Interface de gestion des livres	33
6.3	Interface de gestion des emprunts	33
6.4	Tableau de bord et statistiques	34
6.5	Menus, dialogues et notifications	35
6.6	Résultat obtenu après export CSV	36

Liste des tableaux

4.1	Configuration du projet et base de données	18
4.2	Modèles et couche DAO	19
4.3	Logique métier et validation	20
4.4	Application principale et navigation	21
4.5	Interfaces de gestion des livres et des emprunts	22
4.6	Tableau de bord et statistiques	22
4.7	Écrans complémentaires et personnalisation	23
4.8	Export, notifications et tests fonctionnels	23

Introduction

L'évolution des technologies de l'information a considérablement transformé les méthodes de gestion des données dans de nombreux domaines. Dans le secteur des bibliothèques, la gestion manuelle des livres et des emprunts peut rapidement devenir complexe, surtout lorsque le volume des informations augmente. Il devient alors nécessaire de disposer d'une application informatique permettant d'automatiser ces tâches, d'améliorer l'organisation des données et de faciliter leur consultation.

Dans ce contexte, nous avons développé **SmartLibrary Pro**, une application de gestion de bibliothèque réalisée dans le cadre du module *Développement Java IHM*. Cette application a été conçue à l'aide de **JavaFX** pour la création de l'interface graphique, de **MySQL** pour le stockage des données et de **JDBC** pour assurer la communication entre l'application et la base de données.

L'objectif principal de ce projet est de permettre la gestion efficace des livres et des emprunts à travers une interface moderne, ergonomique et intuitive. L'application offre plusieurs fonctionnalités telles que l'ajout, la modification et la suppression des données, la recherche et le filtrage des informations, l'affichage de statistiques, ainsi que l'exportation des données au format CSV.

Sur le plan technique, le projet repose sur une architecture **MVC (Modèle – Vue – Contrôleur)** associée au patron de conception **DAO (Data Access Object)**, garantissant une bonne organisation du code, une meilleure maintenabilité et une séparation claire des responsabilités.

Ce rapport présente les différentes étapes de réalisation du projet, depuis l'analyse des besoins et la conception de l'application jusqu'à son implémentation et la présentation de ses principales fonctionnalités.

Chapitre 1

Présentation du projet

1.1 Description de l'application

Dans un contexte où la numérisation des services devient indispensable, les bibliothèques ont besoin d'outils informatiques performants permettant d'assurer une gestion efficace de leurs ressources. La gestion manuelle des livres et des emprunts présente plusieurs limites, notamment le risque d'erreurs, la difficulté de suivi des disponibilités et la lenteur de traitement des opérations courantes.

Afin de répondre à ces besoins, nous avons développé **SmartLibrary Pro**, une application de gestion de bibliothèque moderne conçue à l'aide de **JavaFX** pour l'interface graphique, **MySQL** pour la gestion des données et **JDBC** pour la communication avec la base de données.

L'application permet de gérer efficacement les livres disponibles dans la bibliothèque ainsi que les emprunts effectués par les utilisateurs. Elle offre une interface ergonomique et intuitive permettant d'effectuer rapidement les différentes opérations de gestion tout en assurant la fiabilité des données.

Par ailleurs, SmartLibrary Pro intègre plusieurs fonctionnalités avancées telles que la recherche instantanée, le filtrage des données, l'affichage de statistiques sous forme de graphiques, la personnalisation de l'interface et l'exportation des données au format CSV.

Les deux principales entités gérées par l'application sont :

- **Livre** : représente les ouvrages disponibles dans la bibliothèque.
- **Emprunt** : représente les opérations d'emprunt associées aux livres.

L'ensemble du projet a été développé selon l'architecture **MVC (Modèle - Vue - Contrôleur)** en utilisant le patron de conception **DAO (Data Access Object)** afin de garantir une bonne organisation du code et une maintenance simplifiée.

1.2 Besoins fonctionnels

L'application SmartLibrary Pro doit répondre aux besoins fonctionnels suivants :

- Ajouter, modifier, supprimer et consulter les informations relatives aux livres.

- Gérer les emprunts de livres avec enregistrement des dates d'emprunt et de retour.
- Assurer la mise à jour automatique du stock lors des opérations d'emprunt et de restitution.
- Afficher les données sous forme de tableaux interactifs grâce au composant TableView.
- Effectuer des recherches instantanées sur les livres à partir du titre, de l'auteur ou de l'ISBN.
- Filtrer les données selon différents critères tels que la catégorie ou la disponibilité.
- Générer des statistiques permettant de visualiser l'état général de la bibliothèque.
- Exporter les données au format CSV pour faciliter leur exploitation externe.
- Permettre la personnalisation de l'interface utilisateur (thème, couleurs et paramètres d'affichage).
- Afficher des notifications et des messages de confirmation lors des différentes opérations.
- Fournir une aide intégrée facilitant l'utilisation de l'application.

1.3 Besoins non fonctionnels

Outre les fonctionnalités métier, l'application doit satisfaire plusieurs exigences de qualité :

- **Ergonomie** : proposer une interface moderne, intuitive et facile à utiliser.
- **Performance** : assurer une exécution rapide des opérations de recherche, de filtrage et d'accès à la base de données.
- **Fiabilité** : garantir l'intégrité des données stockées dans la base MySQL.
- **Maintenabilité** : adopter une architecture logicielle claire basée sur les modèles MVC et DAO.
- **Sécurité** : contrôler la validité des données saisies afin d'éviter les incohérences.
- **Portabilité** : permettre l'exécution de l'application sur tout système disposant d'une machine virtuelle Java compatible.
- **Évolutivité** : faciliter l'ajout futur de nouvelles fonctionnalités sans modification importante de l'architecture existante.
- **Robustesse** : gérer correctement les erreurs liées aux saisies utilisateur ou à la connexion avec la base de données.
- **Accessibilité** : offrir des options de personnalisation de l'interface afin d'améliorer le confort d'utilisation.

Chapitre 2

Conception

2.1 Architecture MVC

Afin d'assurer une bonne organisation du code et de faciliter la maintenance de l'application, le projet **SmartLibrary Pro** a été développé selon le modèle architectural **MVC (Modèle – Vue – Contrôleur)**.

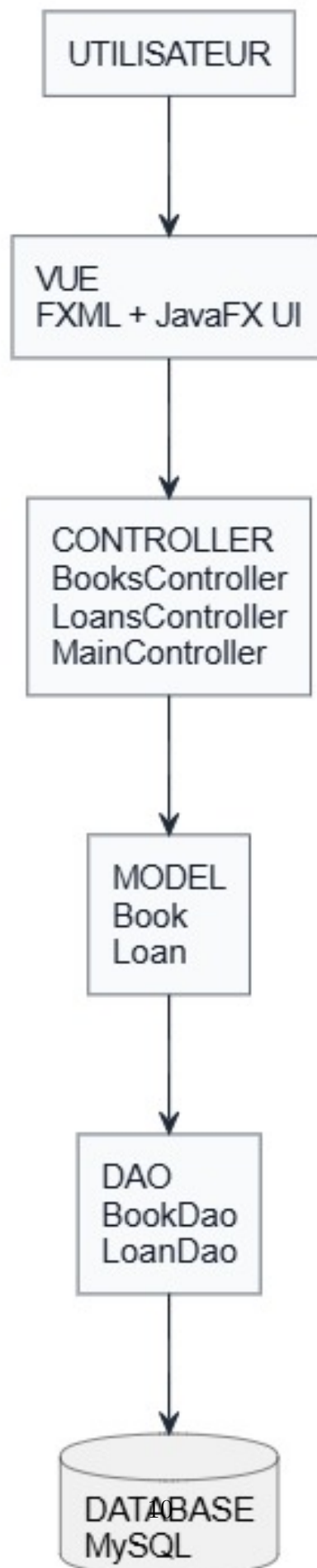
Cette architecture repose sur une séparation claire des responsabilités :

- **Modèle (Model)** : représente les données métier de l'application ainsi que les objets manipulés. Dans notre projet, les principales classes du modèle sont *Book* et *Loan*, qui représentent respectivement les livres et les emprunts.
- **Vue (View)** : correspond à l'interface graphique développée avec JavaFX et décrite à travers les fichiers FXML. Elle permet l'interaction entre l'utilisateur et l'application.
- **Contrôleur (Controller)** : assure la communication entre la vue et le modèle. Les contrôleurs récupèrent les actions de l'utilisateur, exécutent les traitements nécessaires et mettent à jour l'interface.

En complément du modèle MVC, nous avons utilisé le patron de conception **DAO (Data Access Object)** afin de séparer la logique d'accès aux données de la logique métier. Cette approche facilite la gestion de la base de données MySQL et améliore la réutilisabilité du code.

L'organisation générale du projet est illustrée dans la figure suivante.

SmartLibrary Pro - Architecture MVC avec DAO



2.2 Diagramme de classes UML

Le diagramme de classes UML présente les principales classes composant l'application ainsi que les relations existantes entre elles.

Les entités principales du système sont :

- **Book** : représente un livre enregistré dans la bibliothèque.
- **Loan** : représente une opération d'emprunt associée à un livre.

Le diagramme inclut également les classes DAO responsables de l'accès aux données ainsi que les contrôleurs chargés de la gestion de l'interface utilisateur.

La figure suivante présente le diagramme de classes UML du projet.

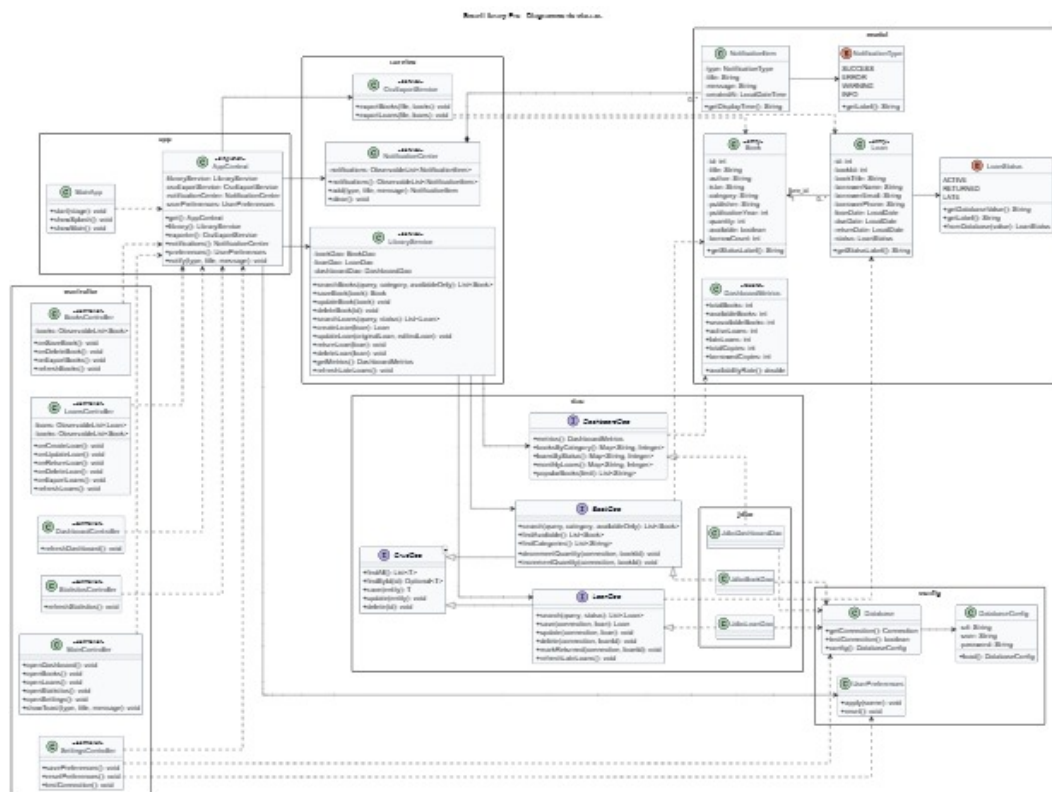


FIGURE 2.2 – Diagramme de classes UML

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet d'identifier les interactions entre l'utilisateur et le système.

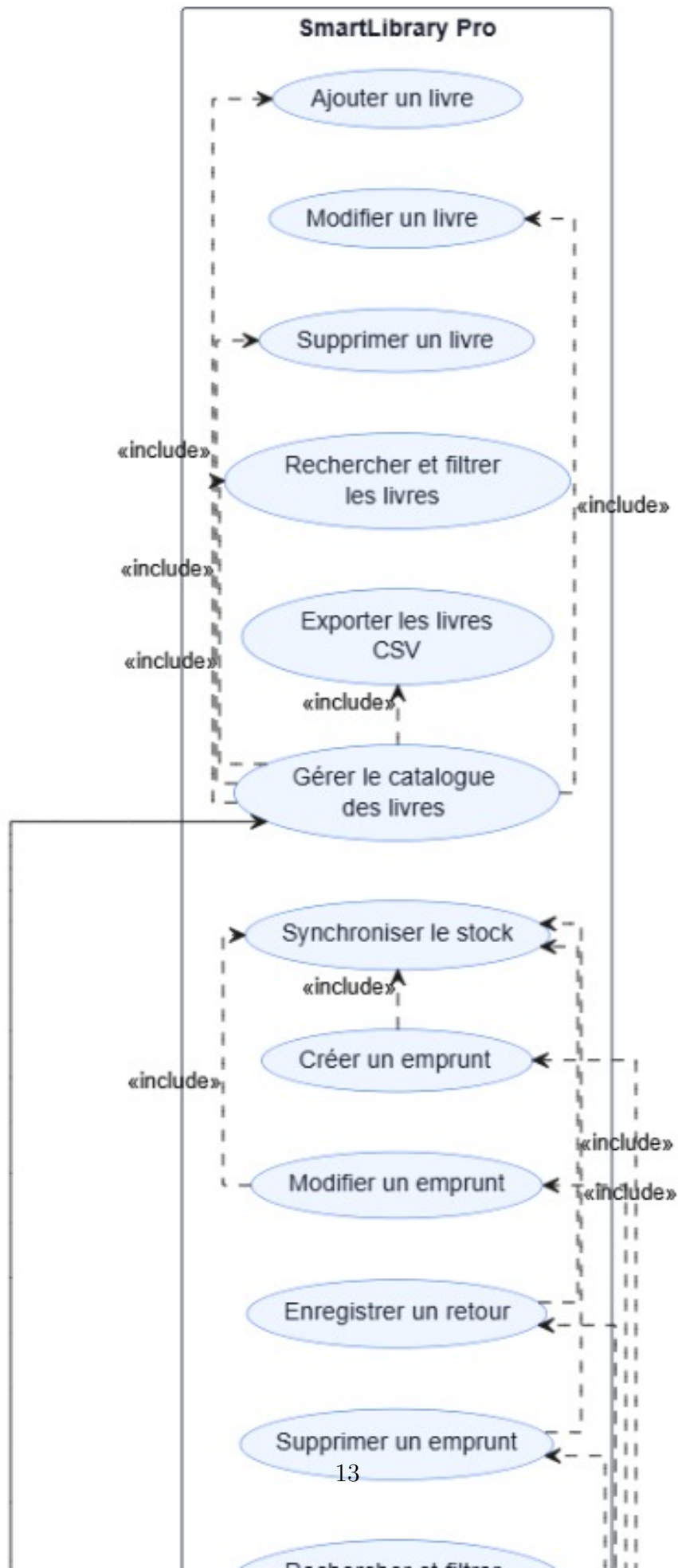
Dans notre application, l'utilisateur peut effectuer plusieurs opérations :

- Gérer les livres (ajout, modification, suppression et consultation).
- Gérer les emprunts.
- Rechercher et filtrer les données.
- Consulter les statistiques.

- Exporter les données au format CSV.
- Personnaliser les paramètres de l'application.

Le diagramme suivant illustre les différents cas d'utilisation du système.

SmartLibrary Pro - Diagramme de cas d'utilisation



2.4 Diagramme de séquence

Le diagramme de séquence permet de représenter les échanges entre les différentes couches de l'application lors de l'exécution d'une fonctionnalité.

Dans notre projet, nous avons choisi d'illustrer le processus d'ajout d'un nouveau livre. Lors de cette opération, l'utilisateur saisit les informations du livre dans l'interface graphique. Le contrôleur récupère ces données, effectue les vérifications nécessaires puis transmet la demande à la couche DAO qui interagit avec la base de données MySQL afin d'enregistrer les informations.

La figure suivante présente le diagramme de séquence correspondant à cette fonctionnalité.

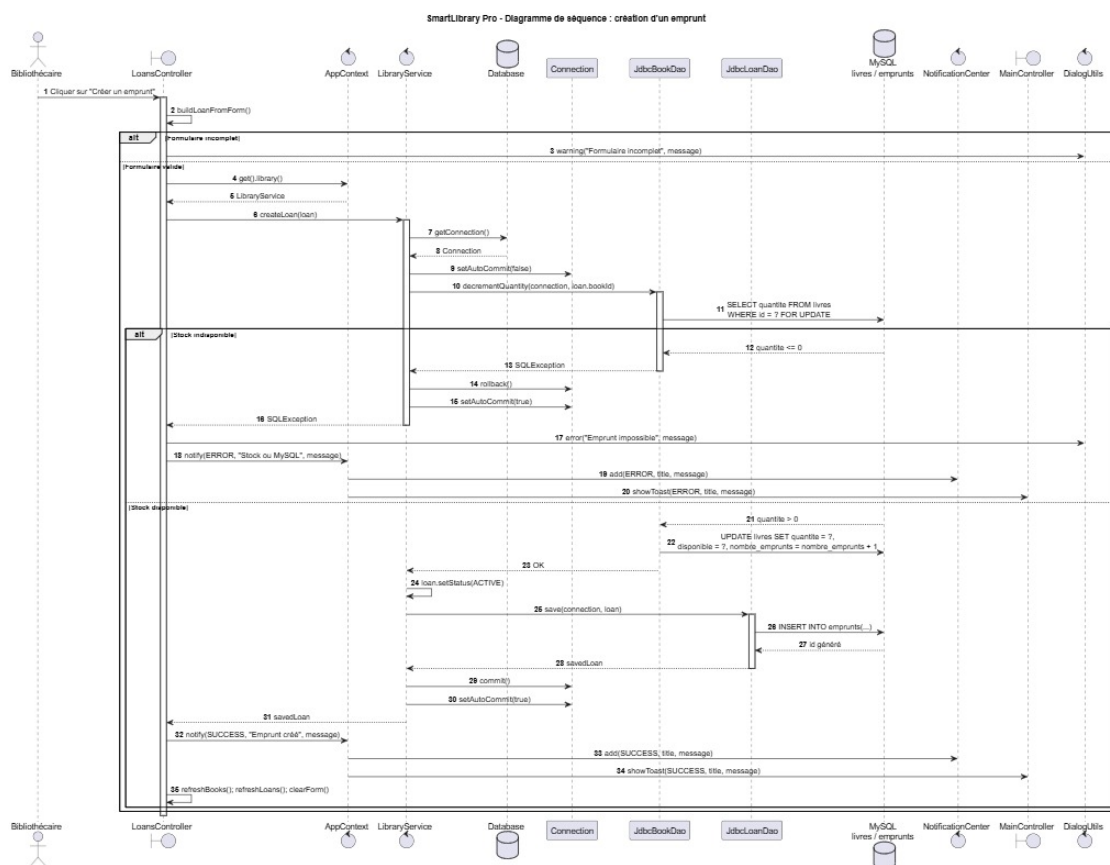


FIGURE 2.4 – Diagramme de séquence : ajout d'un livre

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

Le projet **SmartLibrary Pro** a été développé dans un environnement orienté Java, avec une architecture basée sur le modèle **MVC** et le patron **DAO**. L'application est une solution de gestion de bibliothèque permettant de gérer les livres, les emprunts, les retours, les statistiques et les notifications.

Les principaux outils et technologies utilisés sont les suivants :

- **Langage de programmation** : Java.
- **Version Java** : Java 21, configuré dans le fichier `pom.xml`.
- **Interface graphique** : JavaFX 21.0.5 avec FXML.
- **Architecture** : MVC avec séparation entre les vues, contrôleurs, modèles, services et DAO.
- **Gestion de projet** : Maven.
- **Base de données** : MySQL.
- **Connexion à la base de données** : JDBC avec MySQL Connector/J 9.1.0.
- **IDE utilisé** : IntelliJ IDEA.
- **Modélisation UML** : PlantUML, utilisable avec Visual Studio Code.
- **Feuilles de style** : CSS JavaFX pour la personnalisation de l'interface.

Le fichier `pom.xml` centralise les dépendances du projet, notamment JavaFX, MySQL Connector/J, ainsi que les plugins Maven nécessaires à la compilation et au lancement de l'application.

3.2 Structure du projet

Le projet est organisé selon une structure claire afin de séparer les responsabilités de chaque partie de l'application. Les fichiers Java sont placés dans le répertoire `src/main/java`, tandis que les fichiers FXML, CSS et de configuration sont placés dans `src/main/resources`.

Listing 3.1 – Structure du projet SmartLibrary Pro

```

1 SmartLibraryPro/
2 |-- pom.xml
3 |-- run.bat
4 |-- database/
5 |   |-- init_db.sql
6 |-- src/
7 |   |-- main/
8 |     |-- java/
9 |       |-- module-info.java
10 |       |-- com/
11 |         |-- smartlibrary/
12 |           |-- app/
13 |             |-- config/
14 |               |-- controller/
15 |                 |-- dao/
16 |                   |-- jdbc/
17 |                     |-- model/
18 |                       |-- service/
19 |                         |-- util/
20 |       |-- resources/
21 |         |-- database.properties
22 |         |-- com/
23 |           |-- smartlibrary/
24 |             |-- view/
25 |               |-- style/

```

Le rôle des principaux répertoires est le suivant :

- `app` : contient les classes principales de l'application, notamment `MainApp` pour le lancement et `AppContext` pour le contexte global.
- `config` : contient la configuration de la base de données et les préférences utilisateur.
- `controller` : contient les contrôleurs JavaFX liés aux fichiers FXML, comme `BooksController`, `LoansController` et `MainController`.
- `model` : contient les classes métier de l'application, notamment `Book`, `Loan`, `LoanStatus` et `DashboardMetrics`.
- `dao` : contient les interfaces d'accès aux données, comme `BookDao`, `LoanDao` et `DashboardDao`.
- `dao/jdbc` : contient les implémentations JDBC des DAO permettant la communication avec MySQL.
- `service` : contient la logique métier principale, comme la gestion des emprunts,

la synchronisation du stock, l'export CSV et les notifications.

- `util` : regroupe les classes utilitaires pour la validation, les boîtes de dialogue et les animations.
- `resources/view` : contient les interfaces graphiques FXML de l'application.
- `resources/style` : contient le fichier CSS utilisé pour styliser l'interface JavaFX.
- `database` : contient le script SQL `init_db.sql` permettant de créer la base de données MySQL et ses tables.

Chapitre 4

Répartition des tâches

Le développement de l'application **SmartLibrary Pro** a été réparti entre les deux membres du groupe de manière équilibrée. Afin d'éviter les tableaux trop longs et d'améliorer la lisibilité du rapport, les tâches sont organisées par blocs fonctionnels : configuration, base de données, couche métier, accès aux données, interface graphique, services complémentaires et tests.

4.1 Tâches de l'étudiant 1 – Abdeljalil ELGADI

L'étudiant 1 a principalement travaillé sur la partie technique interne de l'application : configuration du projet, base de données, modèles, DAO JDBC et logique métier.

Configuration du projet et base de données

Tâche réalisée	Fichier(s) concerné(s)	Statut
Mise en place de la structure Maven du projet et configuration des dépendances JavaFX, MySQL et du module Java.	<code>pom.xml</code> , <code>module-info.java</code>	Réalisé
Création de la base de données MySQL avec les tables livres et emprunts, les contraintes, les clés étrangères, les index et les données de démonstration.	<code>database/init_db.sql</code>	Réalisé
Configuration des paramètres JDBC utilisés par l'application pour se connecter à la base <code>smart_library</code> .	<code>database.properties</code>	Réalisé
Développement de la classe d'accès à MySQL et du chargement dynamique des paramètres de connexion.	<code>Database.java</code> , <code>DatabaseConfig.java</code>	Réalisé

TABLE 4.1 – Configuration du projet et base de données

Modèles et accès aux données

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception des classes modèles représentant les entités principales de l'application : livre, emprunt, statut d'emprunt et indicateurs du tableau de bord.	Book.java, Loan.java, LoanStatus.java, DashboardMetrics.java	Réalisé
Création des interfaces DAO afin de séparer les requêtes SQL du reste de l'application.	CrudDao.java, BookDao.java, LoanDao.java, DashboardDao.java	Réalisé
Implémentation JDBC des opérations sur les livres : ajout, modification, suppression, recherche, filtrage, catégories et gestion du stock.	JdbcBookDao.java	Réalisé
Implémentation JDBC des opérations sur les emprunts : création, modification, retour, suppression, recherche par statut et actualisation des retards.	JdbcLoanDao.java	Réalisé

TABLE 4.2 – Modèles et couche DAO

Logique métier et règles de gestion

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement des requêtes statistiques : métriques globales, livres par catégorie, emprunts par statut, évolution mensuelle et livres populaires.	JdbcDashboardDao.java	Réalisé
Développement du service métier central assurant la cohérence entre les emprunts et le stock à l'aide de transactions SQL.	LibraryService.java	Réalisé
Mise en place des validations utilisées dans les formulaires : champs obligatoires et contrôle de l'année de publication.	Validators.java	Réalisé

TABLE 4.3 – Logique métier et validation

4.2 Tâches de l'étudiant 2 – Hicham Abadour

L'étudiant 2 a principalement travaillé sur la partie interface utilisateur : fenêtres FXML, contrôleurs JavaFX, navigation, tableaux de bord, export, notifications, préférences et tests fonctionnels.

Application principale et navigation

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement de la classe principale de lancement et du contexte global de l'application.	MainApp.java, ApplicationContext.java	Réalisé
Conception de la fenêtre principale avec menu supérieur, barre latérale, zone de contenu et état de connexion MySQL.	main.fxml	Réalisé
Développement du contrôleur principal : navigation entre les pages, raccourcis clavier, activation des boutons et notifications toast.	MainController.java	Réalisé

TABLE 4.4 – Application principale et navigation

Interfaces de gestion

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement de l'interface de gestion des livres : tableau, formulaire, barre de recherche, filtres, boutons d'action et export.	<code>books.fxml</code>	Réalisé
Développement du contrôleur des livres : ajout, modification, suppression, recherche, filtrage, chargement des catégories et export CSV.	<code>BooksController.java</code>	Réalisé
Développement de l'interface de gestion des emprunts : tableau, formulaire, sélection du livre, dates, durée, retour et suppression.	<code>loans.fxml</code>	Réalisé
Développement du contrôleur des emprunts : création, modification, retour, suppression, recherche, filtre par statut et actualisation des listes.	<code>LoansController.java</code>	Réalisé

TABLE 4.5 – Interfaces de gestion des livres et des emprunts

Tableau de bord et statistiques

Tâche réalisée	Fichier(s) concerné(s)	Statut
Réalisation du tableau de bord avec indicateurs clés, graphiques JavaFX, liste des livres populaires et barre de progression du stock.	<code>dashboard.fxml</code> , <code>DashboardController.java</code>	Réalisé
Réalisation de l'interface des statistiques avec graphiques, taux de disponibilité et indicateurs opérationnels.	<code>statistics.fxml</code> , <code>StatisticsController.java</code>	Réalisé

TABLE 4.6 – Tableau de bord et statistiques

Écrans complémentaires et finition

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement des écrans complémentaires : paramètres, notifications, aide, à propos et écran de démarrage.	settings.fxml, notifications.fxml, help.fxml, about.fxml, splash.fxml	Réalisé
Développement des contrôleurs associés aux écrans complémentaires : préférences, notifications, aide, à propos et splash screen.	SettingsController.java, NotificationsController.java, HelpController.java, AboutController.java, SplashController.java	Réalisé
Personnalisation visuelle de l'application : styles CSS, cartes KPI, tableaux, formulaires, boutons, thèmes et animations.	styles.css, Animations.java, UserPreferences.java	Réalisé

TABLE 4.7 – Écrans complémentaires et personnalisation

Export, notifications et tests

Tâche réalisée	Fichier(s) concerné(s)	Statut
Mise en place de l'export CSV pour les livres et les emprunts avec encodage UTF-8 et séparateur point-virgule.	CsvExportService.java	Réalisé
Mise en place des boîtes de dialogue, messages d'erreur, confirmations et centre de notifications.	DialogUtils.java, NotificationCenter.java, NotificationItem.java, NotificationType.java	Réalisé
Tests fonctionnels de l'interface : navigation, CRUD livres, CRUD emprunts, retour de livre, export CSV, préférences et affichage des statistiques.	Ensemble de l'application	Réalisé

TABLE 4.8 – Export, notifications et tests fonctionnels

Chapitre 5

Explication du code

5.1 Connexion à la base de données – `Database.java`

La connexion à la base de données est centralisée dans la classe `Database.java`, située dans le package `com.smartlibrary.config`. Cette classe joue le rôle de point d'accès unique pour obtenir une connexion JDBC vers la base MySQL utilisée par l'application SmartLibrary Pro.

Contrairement à une implémentation qui conserverait une seule connexion statique pendant toute l'exécution, cette classe ouvre une nouvelle connexion lorsque la méthode `getConnection()` est appelée. Ce choix est plus fiable, car il évite de réutiliser une connexion déjà fermée ou devenue invalide. La classe est déclarée `final` et son constructeur est privé, ce qui empêche son instantiation.

Les paramètres de connexion ne sont pas écrits directement dans le code. Ils sont chargés par la classe `DatabaseConfig.java` depuis le fichier `database.properties`. Le projet permet aussi de surcharger ces valeurs avec des variables d'environnement, ce qui rend la configuration plus flexible.

Listing 5.1 – Classe `Database.java`

```
1 package com.smartlibrary.config;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public final class Database {
8     private static final DatabaseConfig CONFIG = DatabaseConfig.
9         load();
10
11     private Database() {
12
13     }
14
15     public static Connection getConnection() throws SQLException {
16         return DriverManager.getConnection(
```

```
15         CONFIG.url(),
16         CONFIG.user(),
17         CONFIG.password()
18     );
19 }
20
21 public static boolean testConnection() {
22     try (Connection ignored = getConnection()) {
23         return true;
24     } catch (SQLException exception) {
25         return false;
26     }
27 }
28
29 public static DatabaseConfig config() {
30     return CONFIG;
31 }
32 }
```

Le fichier `database.properties` contient l'URL JDBC, l'utilisateur et le mot de passe. Dans ce projet, l'URL cible la base `smart_library` sur MySQL :

Listing 5.2 – Configuration `database.properties`

```
1 db.url=jdbc:mysql://localhost:3306/smart_library?useSSL=false&
  allowPublicKeyRetrieval=true&serverTimezone=UTC&useUnicode=true&
  characterEncoding=UTF-8
2 db.user=root
3 db.password=
```

La méthode `testConnection()` est utilisée notamment dans l'interface des paramètres et dans la fenêtre principale pour vérifier si MySQL est disponible. Si la connexion échoue, l'application ne se ferme pas brutalement : elle affiche un état d'erreur ou une notification adaptée.

5.2 Les classes modèles

Les classes modèles représentent les données principales manipulées par l'application. Elles se trouvent dans le package `com.smartlibrary.model`. Elles ne contiennent pas la logique d'accès à la base de données : leur rôle est de transporter et structurer les informations utilisées par les contrôleurs, les services et les DAO.

Book.java

La classe Book représente un livre du catalogue. Elle contient les informations nécessaires à la gestion du stock et à l’affichage dans l’interface : identifiant, titre, auteur, ISBN, catégorie, éditeur, année de publication, description, quantité disponible, disponibilité, nombre d’emprunts et dates de création/modification.

Listing 5.3 – Attributs principaux de Book.java

```
1 public class Book {  
2     private int id;  
3     private String title;  
4     private String author;  
5     private String isbn;  
6     private String category;  
7     private String publisher;  
8     private int publicationYear;  
9     private String description;  
10    private int quantity;  
11    private boolean available;  
12    private int borrowCount;  
13 }
```

La méthode `setQuantity()` met automatiquement à jour l’attribut `available`. Ainsi, lorsqu’un livre possède une quantité supérieure à zéro, il est considéré comme disponible. La méthode `getStatusLabel()` retourne un libellé lisible par l’utilisateur : Disponible ou Indisponible.

Loan.java

La classe Loan représente un emprunt. Elle contient l’identifiant de l’emprunt, l’identifiant du livre, le titre du livre, les informations de l’emprunteur, les dates d’emprunt et de retour, le statut et les notes internes.

Listing 5.4 – Attributs principaux de Loan.java

```
1 public class Loan {  
2     private int id;  
3     private int bookId;  
4     private String bookTitle;  
5     private String borrowerName;  
6     private String borrowerEmail;  
7     private String borrowerPhone;  
8     private LocalDate loanDate;
```

```
9     private LocalDate dueDate;  
10    private LocalDate returnDate;  
11    private LoanStatus status = LoanStatus.ACTIVE;  
12    private String notes;  
13 }
```

Le statut d'un emprunt est représenté par l'énumération `LoanStatus`, qui contient trois valeurs : `ACTIVE`, `RETURNED` et `LATE`. Chaque statut possède une valeur stockée en base de données et un libellé affiché dans l'interface.

Autres modèles

Le modèle `DashboardMetrics` est un record Java utilisé pour regrouper les indicateurs du tableau de bord : nombre total de livres, livres disponibles, livres indisponibles, emprunts actifs, retards, exemplaires disponibles et exemplaires empruntés. Il fournit aussi la méthode `availabilityRate()` pour calculer le taux de disponibilité.

Les classes `NotificationItem` et `NotificationType` servent à gérer l'historique des notifications affichées dans l'application.

5.3 Les classes DAO

Le projet utilise le patron **DAO** (*Data Access Object*) afin de séparer la logique d'accès aux données du reste de l'application. Les contrôleurs ne manipulent pas directement les requêtes SQL. Ils passent par le service métier `LibraryService`, qui utilise ensuite les DAO.

L'interface générique `CrudDao<T>` définit les opérations de base communes :

Listing 5.5 – Interface `CrudDao`

```
1 public interface CrudDao<T> {  
2     List<T> findAll() throws SQLException;  
3     Optional<T> findById(int id) throws SQLException;  
4     T save(T entity) throws SQLException;  
5     void update(T entity) throws SQLException;  
6     void delete(int id) throws SQLException;  
7 }
```

BookDao et JdbcBookDao

`BookDao` étend `CrudDao<Book>` et ajoute des méthodes spécifiques aux livres : recherche par texte, filtre par catégorie, filtre par disponibilité, récupération des catégories, incrémentation et décrémentation du stock.

L'implémentation `JdbcBookDao` exécute les requêtes SQL sur la table `livres`. Elle utilise `PreparedStatement` afin d'éviter les injections SQL et de transmettre proprement les paramètres.

Une méthode importante est `decrementQuantity()`, utilisée lors de la création d'un emprunt. Elle verrouille la ligne du livre avec `SELECT ... FOR UPDATE`, vérifie la quantité disponible, puis met à jour le stock.

LoanDao et JdbcLoanDao

`LoanDao` gère les opérations liées aux emprunts. En plus du CRUD classique, il propose des méthodes pour rechercher les emprunts, enregistrer un retour, supprimer un emprunt dans une transaction et actualiser les retards.

`JdbcLoanDao` travaille avec la table `emprunts`. Pour afficher le titre du livre dans les tableaux, les requêtes effectuent une jointure avec la table `livres`.

DashboardDao et JdbcDashboardDao

`DashboardDao` fournit les données statistiques utilisées par le tableau de bord : métriques globales, répartition par catégorie, répartition des statuts d'emprunts, évolution mensuelle et livres populaires.

L'implémentation `JdbcDashboardDao` regroupe ces informations à l'aide de requêtes SQL d'agrégation telles que `COUNT`, `SUM`, `GROUP BY` et `ORDER BY`.

5.4 Les contrôleurs

Les contrôleurs JavaFX sont placés dans le package `com.smartlibrary.controller`. Chaque contrôleur est relié à un fichier FXML grâce à l'attribut `fx:controller`. Les composants graphiques sont injectés dans le contrôleur avec l'annotation `@FXML`.

L'application suit une organisation claire : les vues FXML décrivent l'interface, les contrôleurs gèrent les actions utilisateur, puis le service `LibraryService` applique les règles métier et communique avec les DAO.

MainController

`MainController` contrôle la fenêtre principale. Il gère la navigation entre les pages FXML, l'état de connexion MySQL, les raccourcis clavier et les notifications de type toast. Les méthodes comme `openDashboard()`, `openBooks()` et `openLoans()` chargent dynamiquement les vues dans le conteneur central.

BooksController

BooksController gère l'écran des livres. Il initialise le tableau, configure le formulaire, applique les filtres et traite les actions d'ajout, de modification, de suppression et d'export CSV. La méthode `buildBookFromForm()` construit un objet `Book` à partir des champs de saisie, tout en vérifiant les champs obligatoires avec la classe `Validators`.

LoansController

LoansController gère les emprunts. Il permet de créer un emprunt, le modifier, enregistrer un retour, supprimer un emprunt et exporter la liste en CSV. Il utilise un `ComboBox<Book>` pour sélectionner le livre, des `DatePicker` pour les dates et un `Spinner` pour calculer la durée de l'emprunt.

DashboardController et StatisticsController

DashboardController affiche les indicateurs principaux : total des livres, livres disponibles, livres indisponibles, emprunts actifs, retards et stock. Il remplit aussi des graphiques JavaFX : `PieChart`, `BarChart` et `LineChart`.

StatisticsController fournit une page d'analyse plus détaillée avec les catégories, les statuts des emprunts, l'évolution mensuelle et des indicateurs opérationnels.

Autres contrôleurs

SettingsController gère les préférences utilisateur : thème clair/sombre, couleur d'accent, taille de police, animations et test de connexion MySQL. NotificationsController affiche l'historique des notifications. SplashController, HelpController et AboutController complètent l'expérience utilisateur.

5.5 Les fichiers FXML

Les fichiers FXML sont placés dans `src/main/resources/com/smartlibrary/view`. Ils décrivent la structure graphique des écrans sans mélanger le code Java et la présentation.

Chaque fichier FXML indique son contrôleur avec `fx:controller`. Par exemple, le fichier `books.fxml` est relié à `BooksController`. Les composants graphiques qui doivent être manipulés dans le code possèdent un identifiant `fx:id`.

Listing 5.6 – Exemple de liaison FXML avec un contrôleur

```
1 <BorderPane xmlns="http://javafx.com/javafx/21"
```

```

2         xmlns:fx="http://javafx.com/fxml/1"
3         fx:controller="com.smartlibrary.controller.
           BooksController"
4         styleClass="page">
5
6         <TextField fx:id="searchField"
           promptText="Rechercher titre, auteur, ISBN..." />
7
8
9         <Button text="Enregistrer"
           onAction="#onSaveBook" />
10
11 </BorderPane>

```

Dans cet exemple, `searchField` est injecté dans le contrôleur grâce à `@FXML`. Le bouton déclenche la méthode `onSaveBook()` lorsque l'utilisateur clique dessus.

Les principaux fichiers FXML sont :

- `main.fxml` : structure principale avec menu, barre latérale, zone centrale et notifications.
- `dashboard.fxml` : tableau de bord avec cartes KPI et graphiques.
- `books.fxml` : gestion des livres avec tableau, filtres et formulaire.
- `loans.fxml` : gestion des emprunts avec tableau, formulaire, dates et actions de retour.
- `statistics.fxml` : statistiques détaillées.
- `settings.fxml` : préférences visuelles et test de connexion.
- `notifications.fxml` : historique des notifications.

5.6 Fonctionnalité spécifique au projet

La fonctionnalité la plus importante du projet est la **synchronisation automatique du stock lors des emprunts et des retours**. Lorsqu'un emprunt est créé, le stock du livre diminue automatiquement. Lorsqu'un livre est retourné, le stock augmente automatiquement. Cette logique évite les incohérences entre le catalogue et les emprunts.

Cette fonctionnalité est implémentée dans `LibraryService`. Elle utilise une transaction SQL pour garantir que la modification du stock et l'enregistrement de l'emprunt sont validés ensemble. En cas d'erreur, un `rollback` annule l'opération.

Listing 5.7 – Création d'un emprunt avec transaction

```

1 public Loan createLoan(Loan loan) throws SQLException {
2     try (Connection connection = Database.getConnection()) {
3         try {
4             connection.setAutoCommit(false);

```

```
5         bookDao.decrementQuantity(connection, loan.getBookId())
6         ;
7         loan.setStatus(LoanStatus.ACTIVE);
8         Loan savedLoan = loanDao.save(connection, loan);
9         connection.commit();
10        return savedLoan;
11    } catch (SQLException exception) {
12        connection.rollback();
13        throw exception;
14    } finally {
15        connection.setAutoCommit(true);
16    }
17 }
```

Le retour d'un livre suit la même logique : l'application incrémente le stock puis marque l'emprunt comme retourné. La suppression d'un emprunt actif ou en retard restaure également le stock. Le projet gère aussi l'actualisation automatique des retards grâce à la méthode `refreshLateLoans()`.

Chapitre 6

Interfaces de l'application

6.1 Fenêtre principale

La fenêtre principale est définie dans le fichier `main.fxml` et contrôlée par `MainController`. Elle utilise une structure `BorderPane` avec un menu supérieur, une barre latérale de navigation et une zone centrale dans laquelle les différentes pages sont chargées dynamiquement.

La barre latérale donne accès aux principaux modules : tableau de bord, livres, emprunts, statistiques, notifications, paramètres, aide et à propos. La fenêtre affiche aussi l'état de connexion MySQL et un conteneur de notifications toast.



FIGURE 6.1 – Fenêtre principale de l'application

6.2 Interfaces de gestion de l'entité 1

La première entité principale est le **Livre**. L'interface de gestion des livres est définie dans `books.fxml`. Elle permet de consulter le catalogue, rechercher un livre, filtrer par catégorie et disponibilité, ajouter un nouveau livre, modifier un livre existant, supprimer un livre et exporter les données au format CSV.

L'écran contient un tableau affichant l'ID, le titre, l'auteur, la catégorie, la quantité

et le statut. À droite, un formulaire permet de saisir les informations du livre : titre, auteur, ISBN, catégorie, éditeur, année de publication, quantité et description.

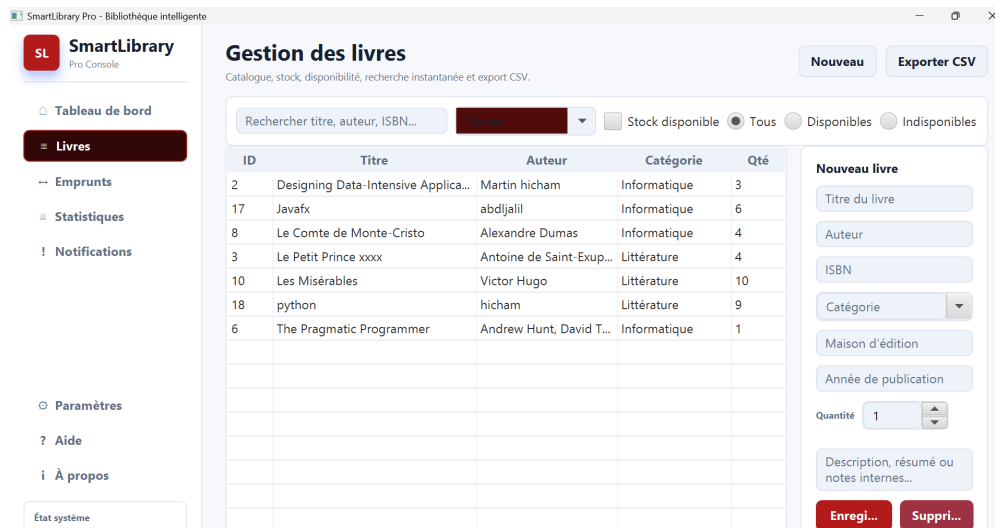


FIGURE 6.2 – Interface de gestion des livres

6.3 Interfaces de gestion de l'entité 2

La deuxième entité principale est l'**Emprunt**. L'interface correspondante est définie dans `loans.fxml`. Elle permet de créer, modifier, supprimer et clôturer un emprunt par retour de livre.

L'utilisateur sélectionne un livre, renseigne les informations de l'emprunteur, choisit la date d'emprunt et la date de retour prévue. Un champ de durée permet de calculer automatiquement la date de retour prévue. Le tableau affiche les emprunts avec le livre concerné, l'emprunteur, la date de retour prévue et le statut.

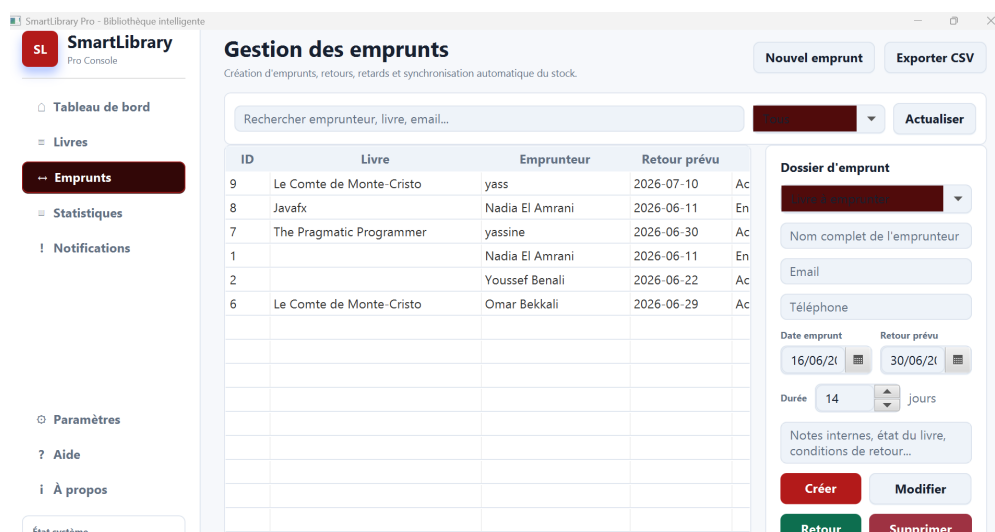


FIGURE 6.3 – Interface de gestion des emprunts

6.4 Tableau de bord – Statistiques

Le tableau de bord est défini dans `dashboard.fxml`. Il présente une vue synthétique de la bibliothèque à travers plusieurs indicateurs : nombre total de livres, livres disponibles, livres indisponibles, emprunts actifs, retards et progression du stock.

L'écran contient également des graphiques JavaFX :

- un `PieChart` pour la répartition des emprunts par statut ;
- un `BarChart` pour les catégories du catalogue ;
- un `LineChart` pour l'évolution mensuelle des emprunts ;
- une liste des livres les plus populaires.

La page `statistics.fxml` complète cette analyse avec des graphiques dédiés aux catégories, aux statuts des emprunts, à l'évolution mensuelle et au taux de disponibilité.

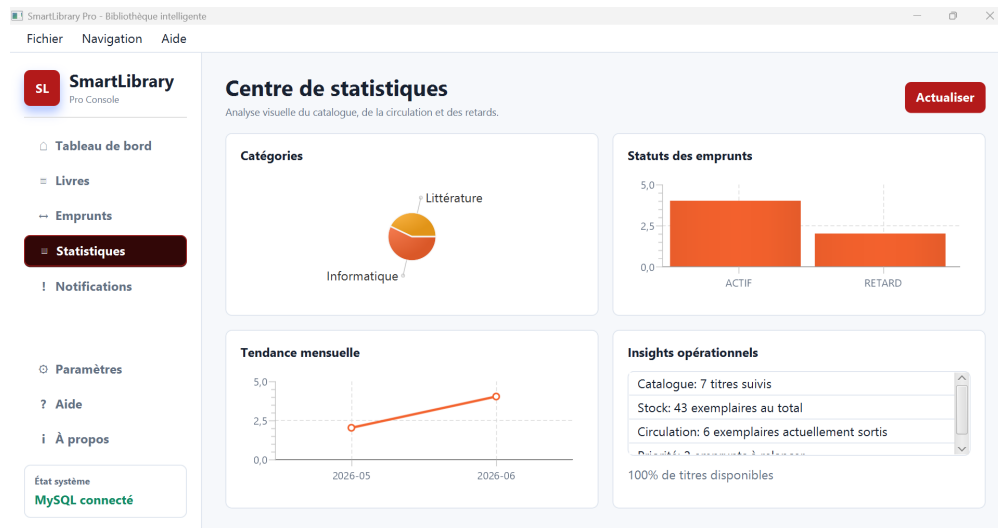


FIGURE 6.4 – Tableau de bord et statistiques

6.5 Menus et dialogues

L'application propose deux systèmes de navigation : un menu supérieur et une barre latérale. Le menu supérieur contient les menus **Fichier**, **Navigation** et **Aide**. La barre latérale permet d'accéder rapidement aux modules les plus utilisés.

Les boîtes de dialogue sont centralisées dans la classe `DialogUtils`. Cette classe propose des méthodes pour afficher des messages d'information, d'erreur, d'avertissement et de confirmation. Les confirmations sont utilisées avant les opérations sensibles comme la suppression d'un livre, la suppression d'un emprunt ou le retour d'un livre.

L'application utilise aussi des notifications toast via `NotificationCenter` et `MainController`. Ces notifications informent l'utilisateur après les opérations réussies ou lors d'une erreur MySQL.

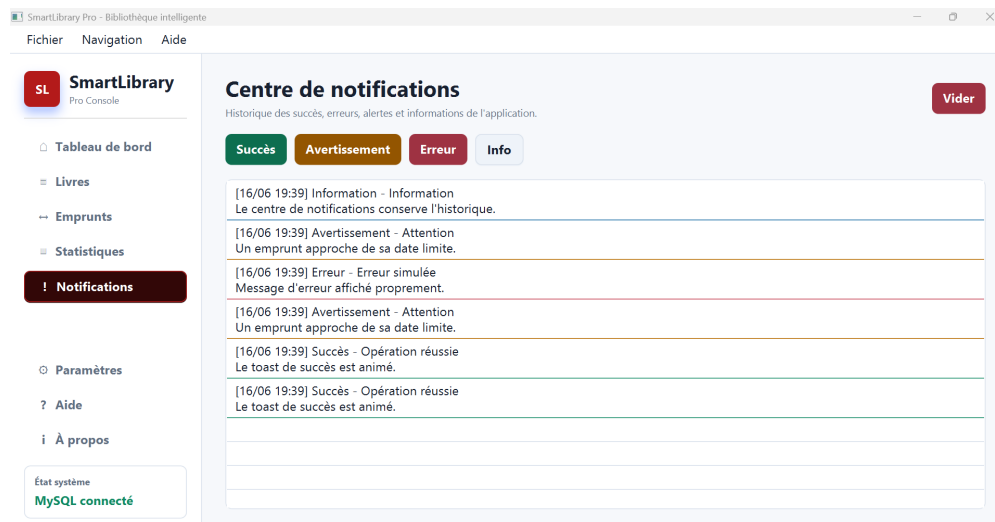


FIGURE 6.5 – Menus, dialogues et notifications

6.6 Export de données

SmartLibrary Pro permet l'export des livres et des emprunts au format **CSV**. Cette fonctionnalité est implémentée dans la classe `CsvExportService`. L'export utilise l'encodage UTF-8 et le séparateur point-virgule, ce qui rend les fichiers exploitables dans des tableurs comme Excel ou LibreOffice Calc.

Depuis l'interface des livres, le bouton **Exporter CSV** génère un fichier contenant les colonnes : identifiant, titre, auteur, ISBN, catégorie, éditeur, année, quantité, statut et nombre d'emprunts.

Depuis l'interface des emprunts, l'export contient les colonnes : identifiant, livre, emprunteur, email, téléphone, date d'emprunt, date de retour prévue, date de retour effective, statut et notes.

Listing 6.1 – Export CSV des livres

```
1 writer.write("id;titre;auteur;isbn;categorie;editeur;annee;quantite  
;statut;emprunts");
```

Enregistrement automatique livres-smartli... • Enregistré dans ce PC Rechercher

Fichier Accueil Insertion Dessin Mise en page Formules Données Révision Affichage Automatiser Aide Acrobat

Coller Aptos Narrow 11 A⁺ A⁻ G I Standard 000 Mise en forme conditionnelle Insérer Supprimer Format

Presse-papiers Police Alignement Nombre Styles Styles de cellules Cellules

PERTE DE DONNÉES POTENTIELLE Vous risquez de perdre certaines fonctionnalités si vous enregistrez ce classeur au format csv (délimité par des virgules). Pour conserver ces fonctionnalités, enregistrez-le dans un format de fichier Excel.

A1 id

	A	B	C	D	E	F	G	H	I	J	K	L
	id	titre	auteur	isbn	categorie	editeur	annee	quantite	statut	emprunts		
2		2 Designing Da	Martin hichar	9,7814E+12	Informatique	O'Reilly	2017	3	Disponible	14		
3		17 Javafx	abdljalil	1234567888	Informatique	Ensao	2005	6	Disponible	1		
4		8 Le Comte de	Alexandre Du	9,7823E+12	Informatique	Le Livre de Po	1844	4	Disponible	13		
5		3 Le Petit Princ	Antoine de Se	9,7821E+12	Littérature	Gallimard	1943	4	Disponible	21		
6		10 Les Misérables	Victor Hugo	9,7823E+12	Littérature	Le Livre de Po	1862	10	Disponible	10		
7		18 python	hicham	123445677	Littérature	Ens	1999	9	Disponible	0		
8		6 The Pragmati	Andrew Hunt,	9,7802E+12	Informatique	Addison-Wes	1999	1	Disponible	17		
9												
10												
11												
12												

FIGURE 6.6 – Résultat obtenu après export CSV

Conclusion

Ce projet a permis de réaliser une application de gestion de bibliothèque complète avec JavaFX, MySQL, JDBC et Maven. SmartLibrary Pro couvre les besoins essentiels d'une bibliothèque : gestion du catalogue, gestion des emprunts, retours de livres, suivi du stock, statistiques, notifications et export CSV.

L'un des points forts du projet est la séparation claire des responsabilités. Les vues FXML définissent l'interface graphique, les contrôleurs traitent les actions utilisateur, les modèles représentent les données, les DAO assurent l'accès à MySQL et le service `LibraryService` centralise les règles métier. Cette organisation rend le code plus lisible, plus maintenable et plus simple à faire évoluer.

La principale difficulté technique a concerné la cohérence du stock lors des emprunts et des retours. Cette difficulté a été résolue grâce à l'utilisation de transactions SQL : une opération d'emprunt modifie le stock et enregistre l'emprunt dans une même transaction. En cas d'erreur, l'opération est annulée afin d'éviter les incohérences.

Comme perspectives d'amélioration, il serait possible d'ajouter un système d'authentification, une gestion des rôles, un module de réservation, des rapports PDF, une sauvegarde automatique de la base de données et des tests unitaires plus complets. Malgré ces pistes d'amélioration, l'application actuelle constitue une base solide et professionnelle pour la gestion d'une bibliothèque.

Références

- Documentation officielle Java : <https://docs.oracle.com/en/java/>
- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Documentation MySQL Connector/J : <https://dev.mysql.com/doc/connector-j/en/>
- Documentation Maven : <https://maven.apache.org/guides/>
- Documentation PlantUML : <https://plantuml.com/fr/>